

Module 1 — Use Case Specifications (Dinuka)

Authentication, Registration and Service Provider Invoice Generation.

Companion to the use case diagram [docs/uml/use-cases/01-dinuka-authentication-invoicing.puml](#) . Every flow below was written against the implemented code, not against the specification document, so the numbered steps match what actually executes.

Actors: Visitor · Client · Service Provider · System Admin · Verification Admin · Email Service (external)

Use case index

ID	Use case	Type	Primary actor
UC-1	Select Registration Role	Primary	Visitor
UC-2	Register as Client	Primary	Visitor, System Admin
UC-3	Register as Service Provider	Primary	Visitor, System Admin
UC-28	Reapply After Rejection	Extends UC-3	Service Provider
UC-4	Upload Verification Documents	Included by UC-3	—
UC-5	Accept Terms and Conditions	Included by UC-3	—
UC-6	Validate Registration Data	Included by UC-2, UC-3	—
UC-7	Generate Unique 6-Character Token	Included by UC-2, UC-3	—
UC-8	Hash and Store Password	Included by UC-2, UC-3, UC-20	—
UC-9	Log In	Primary	All four roles
UC-10	Validate Credentials	Included by UC-9	—
UC-11	Create Authenticated Session	Included by UC-9	—
UC-12	Redirect by Role	Included by UC-9	—
UC-13	Block Banned User	Extends UC-9	—
UC-14	Show Pending Verification Page	Extends UC-12	—
UC-15	Show Rejected Application Page	Extends UC-12	—
UC-16	Log Out	Primary	All four roles
UC-17	Invalidate Session	Included by UC-16	—
UC-18	Request Password Reset	Primary	Visitor
UC-19	Validate Reset Token	Included by UC-20	—
UC-20	Set New Password	Primary	Visitor
UC-21	Generate Invoice	Primary	Service Provider
UC-22	Autofill Job & Payment Details	Included by UC-21	—
UC-23	Lock Amount from Payment Record	Extends UC-21	—
UC-24	Enter Amount Manually	Extends UC-21	—
UC-25	Create Invoice PDF	Included by UC-21	—
UC-26	Download Invoice	Primary	Service Provider, Client, System Admin
UC-27	Send System Email	Included by UC-2, UC-18	Email Service

Part A — Primary use cases

UC-1 — Select Registration Role

Field	Detail
Primary actor	Visitor
Goal	Choose whether to register as a Client or as a Service Provider
Preconditions	The visitor is not logged in
Trigger	Visitor clicks Sign Up in the public navigation or on the login page
Relationships	Extends into UC-2 and UC-3
Implementation	<code>features/auth/pages/RegistrationRolePage.jsx</code>

Main success scenario

1. The system displays two role cards: *I'm a Client* and *I'm a Service Provider*, each with a short description of what that role can do.
2. The visitor selects one card.
3. The system navigates to the corresponding registration page — UC-2 for Client, UC-3 for Service Provider.

Postconditions

The visitor is on the correct registration form. Nothing is persisted; this use case is purely a routing decision.

UC-2 — Register as Client

Field	Detail
Primary actor	Visitor
Secondary actor	System Admin (creates the account on behalf of a user)
Goal	Create a Client account that can immediately log in and book services
Preconditions	The visitor is not logged in; the chosen username and email are not already registered
Trigger	Visitor submits the Client registration form
Includes	UC-6 Validate Registration Data · UC-7 Generate Unique Token · UC-8 Hash and Store Password · UC-27 Send System Email
Implementation	<code>auth.service.js</code> → <code>registerClient()</code> · <code>POST /api/auth/register/client</code> · <code>ClientRegistrationForm.jsx</code> · admin path: <code>AddClientModal.jsx</code>

Main success scenario

1. The visitor enters username, full name, email, phone, password and confirmation.
2. The visitor pins their home location on the map and enters an address.
3. The system validates every field against `clientRegistrationSchema` (**UC-6**).
4. The system confirms the username is not taken and the email is not registered.
5. The system resolves the `CLIENT` role id from the `roles` table.
6. The system hashes the password with bcrypt at 12 salt rounds (**UC-8**).
7. The system generates a unique 6-character user token (**UC-7**).
8. The system opens a database transaction and inserts the `users` row, the `client_profiles` row and the `client_locations` row, then commits.

9. The system sends a welcome email (**UC-27**).
10. The system writes a `CLIENT_REGISTERED` audit entry.
11. The system returns the new account (HTTP 201) and the visitor is directed to the login page.

Alternate and exception flows

#	Condition	System response
3a	Any field fails validation	422 with the field-level messages; nothing is written
4a	Username already exists	409 "Username is already taken"
4b	Email already registered	409 "Email is already registered"
5a	The <code>CLIENT</code> role row is missing	500 "Client role is not configured"
7a	10 token attempts all collide	500 "Could not generate a unique user token"
8a	Any insert fails	<code>ROLLBACK</code> — no partial account is left behind
9a	The email provider is unreachable	The failure is logged and swallowed; registration still succeeds , because the email is sent after <code>COMMIT</code>
—	The System Admin performs this from User Management	Identical flow; the same route and service are called from <code>AddClientModal.jsx</code>

Postconditions

A `users` row exists with role `CLIENT`, `account_status = 'ACTIVE'` and a unique 6-character token, together with its client profile and primary location. The account can log in immediately — no approval step exists for Clients.

Business rules

- Username 3–30 characters, letters, numbers or underscore.
- Password at least 8 characters containing at least one letter and one digit.
- Phone must match the Sri Lankan format `+94XXXXXXXX` or `0XXXXXXXX`.
- Username and email are stored as `citext`, so uniqueness is case-insensitive.

UC-3 — Register as Service Provider

Field	Detail
Primary actor	Visitor
Secondary actor	System Admin (creates the account on behalf of a user)
Goal	Submit a Service Provider application for verification
Preconditions	The visitor is not logged in; they hold the four required documents
Trigger	Visitor submits step 2 of the Service Provider registration form
Includes	UC-4 Upload Documents · UC-5 Accept Terms · UC-6 Validate · UC-7 Generate Token · UC-8 Hash Password
Implementation	<code>registration.service.js</code> → <code>registerProvider()</code> · <code>POST /api/registration/provider</code> · <code>ProviderRegistrationPage.jsx</code> · admin path: <code>AddProviderModal.jsx</code>

Main success scenario

1. The system loads the reference data — districts and service categories — from `GET /api/registration/reference`.

2. **Step 1:** the visitor enters account details, home district, service district, bio, working hours, hourly charge estimate, and selects one or two service categories.
3. **Step 2:** the visitor enters the police station name and report date, uploads the four documents (**UC-4**), and ticks the Terms and Conditions box (**UC-5**).
4. The system uploads all fields as `multipart/form-data`.
5. Multer stores each file under a UUID filename in private storage, rejecting anything over 5 MB or of the wrong type.
6. The system validates the payload against `providerRegistrationSchema` (**UC-6**), which also asserts `termsAccepted` is literally true.
7. The system confirms all four document fields are present.
8. The system checks the username and email — see UC-3.1 below for the reapplication branch.
9. The system hashes the password (**UC-8**) and generates a unique token (**UC-7**).
10. The system opens a transaction and inserts: the `users` row, the `service_provider_profiles` row, one `provider_service_categories` row per selected category, one `sp_verification_applications` row with `attempt_number = 1`, and four `sp_verification_documents` rows. It commits.
11. The system returns `verificationStatus: PENDING` and the visitor is sent to the Pending Verification page.

Alternate and exception flows

#	Condition	System response
3a	Fewer than four documents attached	422 " <i>field</i> is required"
3b	Terms box not ticked	422 — the schema requires the literal value <code>true</code>
5a	File larger than 5 MB, or a disallowed type	422 "Invalid file type for <i>field</i> ". Only JPG/PNG are accepted; the police report may additionally be a PDF
8a	Username or email belongs to a different account	409 "Username is already taken" / "Email is already registered"
8b	Same identity, previously rejected	Branches to UC-3.1
10a	Any insert fails	ROLLBACK — no partial application
—	Category count exceeds two	Rejected by the schema, and again by the <code>trg_max_two_categories</code> database trigger

Postconditions

A `users` row exists with role `SERVICE_PROVIDER` and `verification_status = 'PENDING'`. **The provider cannot use any provider functionality yet** — they may only reach the Pending Verification page until the Verification Admin approves them.

UC-28 (UC-3.1) — Reapply After Rejection

Field	Detail
Primary actor	Service Provider (rejected)
Goal	Let a rejected Service Provider resubmit a corrected application on the same account
Trigger	The provider clicks Reapply on the Application Rejected page, or resubmits the public provider form with the same credentials
Preconditions	Same username and same email and role <code>SERVICE_PROVIDER</code> and <code>verification_status = 'REJECTED'</code>
Implementation	<code>registration.service.js</code> → <code>resolveReapplicant()</code> , <code>reapplyAsProvider()</code>

Main success scenario

1. The system detects that both the username and the email resolve to the **same** existing account.
2. The system confirms that account is a Service Provider whose status is **REJECTED**.
3. The system verifies the submitted password against the stored hash — proving the applicant owns the account.
4. The system counts prior applications; it must be fewer than **MAX_VERIFICATION_ATTEMPTS** (3).
5. The system opens a transaction: updates the user's name and phone, updates the provider profile, replaces the category rows, inserts a **new** **sp_verification_applications** row with the next attempt number, and inserts four fresh document rows. It commits.
6. The provider returns to **PENDING** and is sent to the Pending Verification page.

Alternate and exception flows

#	Condition	System response
2a	Only one of username/email matches, or the account is a different role, or the status is PENDING / APPROVED	409 — treated as an ordinary duplicate, not a reapplication
3a	Wrong password	409 explaining that this identity belongs to a rejected application, and to either enter the correct password or use Forgot Password
4a	Three attempts already used	409 "This application has already been rejected 3 times and can no longer be resubmitted"

Business rules

- Prior rejected attempts are **never deleted** — the full application history is preserved for the Verification Admin.
- Because the account is never deleted either, once the three attempts are spent that username and email are permanently unusable.

UC-9 — Log In

Field	Detail
Primary actor	Client, Service Provider, System Admin, Verification Admin — one shared path for all four roles
Goal	Obtain an authenticated session and reach the correct area of the system
Preconditions	The account exists
Trigger	The user submits the login form
Includes	UC-10 Validate Credentials · UC-11 Create Authenticated Session · UC-12 Redirect by Role
Extended by	UC-13 Block Banned User
Implementation	auth.service.js → login() · POST /api/auth/login · LoginForm.jsx

Main success scenario

1. The user enters a username **or** email, plus a password.
2. The system looks the account up by either identifier in one query.
3. The system compares the password against the bcrypt hash (**UC-10**).
4. The system confirms the account is not **DEACTIVATED**.
5. The system confirms there is no active ban (**UC-13 does not trigger**).
6. The system stamps **last_login_at**.

- The system inserts an `auth_sessions` row and signs a JWT carrying the user id, role, account status, verification status and the session id as `sid` (**UC-11**).
- The system returns the token and the public user object; the client stores the token.
- The client redirects according to role and verification status (**UC-12**).

Alternate and exception flows

#	Condition	System response
2a	No account matches	401 "Invalid username or password"
3a	Password does not match	401 — the same message as 2a , so an attacker cannot tell which accounts exist
4a	Account is <code>DEACTIVATED</code>	403 "Your account has been deactivated. Please contact support."
5a	An active ban exists	UC-13
—	No attempt limit exists	See the notes at the end of this document

Postconditions

A JWT valid for the configured session lifetime is held by the client, and a matching `auth_sessions` row exists and is not revoked.

UC-16 — Log Out

Field	Detail
Primary actor	Client, Service Provider, System Admin, Verification Admin
Goal	End the current session so the stored token can no longer authenticate
Preconditions	The user is logged in
Trigger	The user clicks Log Out
Includes	UC-17 Invalidate Session
Implementation	<code>auth.service.js</code> → <code>logout()</code> · <code>POST /api/auth/logout</code> · <code>AuthContext.jsx</code>

Main success scenario

- The client calls `POST /api/auth/logout` with the current bearer token.
- `authenticate` verifies the JWT and reads `userId` and `sid` from it.
- The system sets `revoked_at = now()` on that one `auth_sessions` row (**UC-17**).
- The client clears the stored token and returns the user to the public area.

Alternate and exception flows

#	Condition	System response
2a	Token missing, invalid or expired	401 — the client clears the token anyway, so the user still ends up logged out
3a	The token predates session tracking and has no <code>sid</code>	The service returns without doing anything; the token simply expires naturally

Postconditions

`auth_sessions.revoked_at` is set. On the next request `checkAccountStatus` sees the revoked row and rejects the token with 401 — **the JWT is still cryptographically valid, but it no longer authenticates anyone.**

UC-18 — Request Password Reset

Field	Detail
Primary actor	Visitor (a user who cannot log in)
Goal	Receive a link that allows a new password to be set
Preconditions	None — the use case must behave identically whether or not the email exists
Trigger	The visitor submits the Forgot Password form
Includes	UC-27 Send System Email
Implementation	<code>auth.service.js → forgotPassword()</code> · <code>POST /api/auth/forgot-password</code>

Main success scenario

1. The visitor enters their email address.
2. The system looks up the account.
3. The system generates 32 random bytes as the raw token.
4. The system stores the **SHA-256 hash** of that token in `password_reset_tokens` with an expiry timestamp, inside a transaction.
5. The system emails the visitor a link containing the **raw** token (**UC-27**).
6. The system responds: *"If an account with that email exists, a password reset link has been sent."*

Alternate and exception flows

#	Condition	System response
2a	No account with that email	The system returns immediately without creating a token or sending mail — but the response in step 6 is byte-for-byte identical , so the form cannot be used to discover which emails are registered
4a	The insert fails	<code>ROLLBACK</code> , and the error propagates; no email is sent
5a	The email provider is unreachable	Logged and swallowed. The token exists but the user never receives it, and must request a new one

Postconditions

An unused, unexpired `password_reset_tokens` row exists. Only the hash is stored, so a database read alone cannot reset anyone's password.

Business rules

- Requesting a reset does **not** invalidate earlier tokens, and does not lock the account.

UC-20 — Set New Password

Field	Detail
Primary actor	Visitor
Goal	Replace the account password using a valid reset link
Preconditions	The visitor holds a reset link created by UC-18
Trigger	The visitor submits the Reset Password form
Includes	UC-19 Validate Reset Token · UC-8 Hash and Store Password
Implementation	<code>auth.service.js → resetPassword()</code> · <code>POST /api/auth/reset-password</code>

Main success scenario

1. The visitor opens the link; the token is read from the query string.
2. The visitor enters a new password twice.
3. The system validates the password rules and that both entries match.
4. The system hashes the submitted token and looks for a row that is unused and unexpired (**UC-19**).
5. The system opens a transaction, writes the new bcrypt hash to `users.password_hash` (**UC-8**) and sets `used_at` on the token row, then commits.
6. The system confirms success and the visitor is sent to the login page.

Alternate and exception flows

#	Condition	System response
3a	Password too weak or entries differ	422 with the field messages
4a	Token unknown, already used, or expired	400 "This reset link is invalid or has expired" — one message for all three cases
5a	Either statement fails	ROLLBACK — the password is never changed while leaving the token unspent, and the token is never spent without the password changing

Postconditions

The password is replaced and the token is permanently spent.

Known limitation

Resetting the password does **not** revoke existing `auth_sessions` rows. Anyone already logged in on that account stays logged in.

UC-21 — Generate Invoice

Field	Detail
Primary actor	Service Provider
Goal	Produce a permanent PDF invoice for a completed job
Preconditions	The provider is logged in and APPROVED ; the booking is theirs and is COMPLETED ; no invoice exists for it yet
Trigger	The provider clicks Generate Invoice on a row in Completed Jobs
Includes	UC-22 Autofill Job & Payment Details · UC-25 Create Invoice PDF
Extended by	UC-23 Lock Amount (Card) · UC-24 Enter Amount Manually (Cash)
Implementation	<code>invoice.service.js</code> → <code>generateInvoice()</code> · <code>POST /api/provider/invoices/:bookingId</code> · <code>CreateInvoicePage.jsx</code>

Main success scenario

1. The provider opens the invoice form; the system loads the booking and prefills every field (**UC-22**).
2. The amount is either locked (**UC-23**) or entered by the provider (**UC-24**), depending on how the job was paid.
3. The provider submits.
4. The system re-loads the booking and re-checks ownership and **COMPLETED** status — the browser is not trusted.
5. The system confirms no invoice already exists for this booking.

6. The system resolves the final amount: for Card it re-reads the recorded payment and **discards anything the client sent**; for Cash it requires a positive amount in the request.
7. The system writes the PDF to private storage under a UUID filename (**UC-25**).
8. The system inserts the `invoices` row.
9. The system writes an `INVOICE_GENERATED` audit entry and returns the invoice (HTTP 201).

Alternate and exception flows

#	Condition	System response
4a	Booking does not exist	404 "Booking not found"
4b	Booking belongs to another provider	403 "You do not have permission to invoice this booking"
4c	Booking is not <code>COMPLETED</code>	422 "An invoice can only be generated for a completed job"
4d	The provider is not <code>APPROVED</code>	403, from the <code>checkProviderVerification</code> middleware
5a	An invoice already exists	409 "An invoice has already been generated for this job"
6a	Cash job with no amount supplied	422 "Amount is required for a Cash-paid job"
8a	The insert violates the unique constraint (a simultaneous second request)	The written PDF file is deleted , then 409 — no orphan file, still exactly one invoice
8b	Any other insert error	The PDF file is deleted and the error propagates

Postconditions

Exactly one `invoices` row and exactly one PDF file exist for this booking, permanently. The invoice cannot be edited or regenerated.

Business rules

- One invoice per booking, enforced by the `invoices_booking_id_key` unique constraint as well as by the check in step 5.
- `amount > 0`, enforced by the `invoices_amount_check` constraint.
- The invoice number is derived, not stored: `INV-` plus the booking id padded to six digits.

UC-26 — Download Invoice

Field	Detail
Primary actors	Service Provider (own invoice), Client (own booking), System Admin (SP Tracking)
Goal	Retrieve the stored invoice PDF
Preconditions	An invoice has been generated for the booking
Trigger	The actor clicks Download / View Invoice
Implementation	<code>invoice.service.js</code> → <code>getInvoiceDownload()</code> · <code>GET /api/provider/invoices/:bookingId/download</code>

Main success scenario

1. The actor requests the invoice for a booking.
2. `authorizeRoles` admits only `SERVICE_PROVIDER`, `CLIENT` and `SYSTEM_ADMIN`.
3. The system loads the invoice together with the booking's client id.
4. The system confirms the caller is the owning provider, the owning client, or a System Admin.
5. The system confirms the file still exists on disk.

6. The system streams the PDF with a `Content-Disposition` filename of `invoice-<bookingId>.pdf`.

Alternate and exception flows

#	Condition	System response
3a	No invoice for that booking	404 "No invoice has been generated for this job yet"
4a	Caller is none of the three permitted parties	403 "You do not have permission to access this invoice"
5a	The row exists but the file is missing	404 "The invoice file could not be found"

Postconditions

Nothing changes. **Every download returns the identical stored file** — the PDF is never regenerated, so an invoice downloaded today and the same invoice downloaded next year are byte-identical.

Implementation status

The provider path (`features/provider/invoiceApi.js` , reached from Maheli's Completed Jobs) and the client path (`features/client/bookingApi.js` , reached from My Bookings) are both wired up. **The System Admin path is authorised in the backend but has no caller** — nothing in `features/admin` requests `INVOICE_DOWNLOAD` , so the SP Tracking "view invoice" action described in the team responsibility document is not built yet. That UI is Vihas's; Dinuka's endpoint already permits it.

Part B — Included use cases

These never run on their own; they are steps inside the use cases above.

UC-4 — Upload Verification Documents

Included by UC-3. Four documents are mandatory: **face photo, NIC front, NIC back and police report**. Multer writes each to private storage under a generated UUID filename, so a user-supplied filename can never influence the path. Images must be JPG or PNG; the police report may also be a PDF. Each file is capped at 5 MB. One `sp_verification_documents` row is written per file, recording the type, relative storage path, original filename, MIME type and byte size. **Exception:** a missing field yields 422, a bad type or oversized file yields 422 and nothing is stored.

UC-5 — Accept Terms and Conditions

Included by UC-3. The applicant must tick the Terms box before submitting. The schema requires the literal value `true` , so the request cannot be crafted without it. The accepted version (`TERMS_VERSION = 'v1'`) is written onto the `sp_verification_applications` row, so the record shows *which* terms were agreed to, not merely that some were.

UC-6 — Validate Registration Data

Included by UC-2 and UC-3. Zod schemas validate on the server; the same rules are mirrored in `authValidation.js` for immediate feedback in the browser. **The browser copy is a convenience — the server copy is the authority.** Rules: username 3–30 characters of letters, numbers or underscore; valid email; Sri Lankan phone; password of 8+ characters with at least one letter and one digit; passwords must match. Providers additionally: bio 10–1000 characters, working hours 3–500, non-negative hourly estimate, one or two service categories, police station name, report date.

UC-7 — Generate Unique 6-Character Token

Included by UC-2 and UC-3. `generateUserToken()` builds a 6-character string guaranteed to contain at least one uppercase letter, one lowercase letter and one digit, then Fisher–Yates shuffles it so the guaranteed characters are not always in fixed positions. All randomness comes from `crypto.randomInt`. `generateUniqueToken()` retries up to 10 times against the database, then fails with 500 rather than issuing a duplicate. Two database objects back this up: the `uix_users_user_token` unique index and the `chk_token_format` CHECK constraint.

UC-8 — Hash and Store Password

Included by UC-2, UC-3 and UC-20. `bcrypt` with 12 salt rounds, in `utils/passwordUtils.js`. All three callers use the same function. The plaintext password is never written to the database, never logged, and never returned in any response.

UC-10 — Validate Credentials

Included by UC-9. One query resolves the account by username **or** email, joining `roles` and left-joining `service_provider_profiles` for the verification status. `bcrypt.compare` checks the password. A missing account and a wrong password produce the **same** 401 message, deliberately.

UC-11 — Create Authenticated Session

Included by UC-9. An `auth_sessions` row is inserted, then a JWT is signed carrying `userId`, `username`, `role`, `accountStatus`, `verificationStatus` and `sid` — the session row's id. Storing `sid` in the token is what makes an individual session revocable later. `refresh_token_hash` is filled with a random per-session value purely to satisfy its `NOT NULL / UNIQUE` constraints; there is no refresh-token flow.

UC-12 — Redirect by Role

Included by UC-9. Extended by UC-14 and UC-15. `resolveRedirect()` sends the user to `ROLE_HOME_ROUTE`: Client to `/`, Service Provider to `/provider/dashboard`, System Admin to `/admin/system/dashboard`, Verification Admin to `/admin/verification/dashboard`. If the user was bounced to login from a protected page, they return to that page instead. The same rules are enforced again in `ProtectedRoute.jsx`, so **typing another role's URL directly redirects rather than granting access** — the redirect is a security boundary, not just a convenience.

UC-17 — Invalidate Session

Included by UC-16. `revokeAuthSession()` sets `revoked_at` on the one row matching **both** the session id and the user id, so no user can revoke another user's session. Thereafter `checkAccountStatus` — which runs on every authenticated request — sees the revoked row and returns 401.

UC-19 — Validate Reset Token

Included by UC-20. The submitted raw token is SHA-256 hashed and matched against `password_reset_tokens` with `used_at IS NULL AND expires_at > now()`. Unknown, used and expired tokens are all reported with one message so the response reveals nothing about which case applied.

UC-22 — Autofill Job & Payment Details

Included by UC-21. `GET /api/provider/invoices/:bookingId/form` returns everything the invoice needs in a single query joining bookings, both users, the service category, the booking location and the payment record: job description, service category, client name, job location, scheduled window, completion time, payment method and provider name. The

provider types nothing except, for Cash jobs, the amount. The response also carries `amountEditable` and `invoiceExists` so the form renders correctly before anything is submitted.

UC-25 — Create Invoice PDF

Included by UC-21. Built with `pdfkit` in `invoicePdf.js`: a full-width green header band with the HomeHero wordmark, the invoice number, a two-column bordered table of the nine detail rows, the total amount in LKR, a closing line, the provider's name, and the logo. Row heights are measured before the borders are drawn, so a long job description expands its row instead of clipping. The file is written to private storage — **never to a public folder** — under a UUID filename, and is only ever served through UC-26's permission check.

UC-27 — Send System Email

Included by UC-2 and UC-18; the Email Service is an external actor. `sendEmail()` posts to the Resend API. **Every call is wrapped in try/catch and only logs on failure**, and is always invoked *after* the database transaction commits. A mail outage therefore degrades the experience but can never roll back a registration or a reset request. Dinuka's module sends two of these: the Client welcome email and the password reset email.

Part C — Extending use cases

These run only when their condition holds.

UC-13 — Block Banned User

Extends UC-9. Before a session is created, `findActiveBan()` checks `user_bans` for a row with `ban_status = 'ACTIVE'`. If one exists, login fails with 403 and a message naming the reason — and, for a temporary ban, the end date. **The same rule is enforced mid-session:** `checkAccountStatus` re-queries the ban on every authenticated request, so a user banned while logged in is ejected on their next action rather than lingering until the token expires. That path returns 401 rather than 403 so the client's auto-logout interceptor clears the dead token.

UC-14 — Show Pending Verification Page

Extends UC-12. If the user is a Service Provider whose verification status is not `APPROVED` and not `REJECTED`, the redirect goes to the Pending Verification page instead of the provider dashboard. `ProtectedRoute` applies the same rule to direct URL entry, and on the server `checkProviderVerification` returns 403 for any provider-only endpoint. Two named exceptions exist: a pending provider may still browse Explore and public provider profiles.

UC-15 — Show Rejected Application Page

Extends UC-12. If verification status is `REJECTED`, the redirect goes to the Application Rejected page, which explains the outcome and offers the reapplication route described in UC-3.1.

UC-23 — Lock Amount from Payment Record (Card job)

Extends UC-21, when `payment_method = 'CARD'`. The amount is read from the provider earning recorded by the payments module (`getCardPaymentAmount()`), and the form renders it read-only with `amountEditable: false`. On submission the server **re-reads the recorded amount and ignores any value in the request body** — the validation schema documents this explicitly. A tampered browser value therefore cannot alter the invoice. This is the integration point with Visal's Module 4.

UC-24 — Enter Amount Manually (Cash job)

Extends UC-21, when the job was paid in cash. HomeHero never handled the money, so no verified figure exists; the provider types the amount and it must be positive. **This is a self-reported figure and is explicitly not a HomeHero-verified payment record** — the distinction matters, because it is the only place in the system where a money value originates from user input rather than from a recorded transaction.

Notes for the viva

Trust boundaries. Three things a panelist may probe are all answered the same way — the browser is never trusted. Registration validation exists in the browser *and* on the server (UC-6). The invoice amount for a Card job is re-read on the server and the submitted value discarded (UC-23). Role-based redirection exists in the router *and* is re-enforced by route guards and middleware (UC-12).

Where transactions are used, and why. UC-2, UC-3, UC-3.1 and UC-20 each write to more than one table, and each wraps those writes in an explicit transaction with `ROLLBACK` on error. The rule applied throughout is that anything which must be true together is written together — and anything that must *not* be able to undo a commit, such as email (UC-27) or audit logging, happens after it.

Uniform error messages. UC-9 (unknown account vs. wrong password), UC-18 (email exists vs. does not), and UC-19 (unknown vs. used vs. expired token) all deliberately return one message for multiple causes, so the responses cannot be used to enumerate accounts.

The gap to name before you are asked. There is no rate limiting on login (UC-9) or password reset requests (UC-18) — `middleware/rateLimiter.js` is an empty placeholder. bcrypt at 12 rounds makes brute force expensive, but no attempt cap or lockout exists. Naming this yourself, with `express-rate-limit` on those two routes as the fix, reads far better than being caught by it.